Python Engineering - Class 04

Functions and Decomposition | EX09-EX12 - Reusable Engineering Functions

Class objective: Transform monolithic scripts into functions with clear inputs, outputs and responsibilities.

This document is the base exercise guide. It is not a full solution to copy: it organizes the problem, implementation steps, reference values and required validation.

Path

```
01_python_bootcamp/
`-- week_01_logic/class_04_functions/
    |-- notes/
    `-- exercises/
```

Required workflow

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Before coding, write these eight sections. The goal is to separate problem reasoning from syntax.

EX09 - Dynamic Pressure Function

Problem: Encapsulate the dynamic-pressure equation in a reusable function.

Reference inputs: rho, speed

Equations / relationship: q=0.5*rho*speed**2

Expected check: rho=1.225, speed=100 -> 6125 Pa

Implementation guide

- 1. Define parameters.
- 2. Compute q inside the function.
- 3. Return q.
- 4. Call with different values.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX10 - Rocket Equation Function

Problem: Separate validation and Delta_v calculation into functions.

Reference inputs: Isp,g0,m0,mf

Equations / relationship: Delta_v=Isp*g0*ln(m0/mf)

Expected check: Use same valid reference as EX02 and compare result.

Implementation guide

- 1. Create validate_inputs.
- 2. Use Boolean return.

- 3. Compute only after validation.
- 4. Return Delta_v.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX11 - Taylor exp Function

Problem: Convert the Taylor loop into a parameterized function.

Reference inputs: x, N

Equations / relationship: sum x^n/n!

Expected check: Compare against math.exp for several N.

Implementation guide

- 1. Receive x and N.
- 2. Build local accumulation.
- 3. Return approximation.
- 4. Compare several calls.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

EX12 - Vertical Motion with Functions

Problem: Separate validation, Euler step, simulation and analytical solution.

Reference inputs: h0,v0,t0,g,dt

Equations / relationship: Euler + analytical ballistic formulas

Expected check: Functions should return data rather than depend on global variables; analytical solution must not depend on dt.

Implementation guide

- 1. Create validate_inputs().
- 2. Create euler_step().
- 3. Create simulation.
- 4. Create analytical solution.
- 5. Compare results.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

Final validation

Answer these questions as comments at the end of the exercise. The goal is not memorized wording but demonstrating that you can explain the code flow.

- Difference between parameter and argument.
- Difference between print and return.
- What is local scope?
- Why does euler_step return three values?
- Why should the analytical solution not depend on dt?
- What is the advantage of separating validate_inputs from the physical calculation?

Completion criterion

The class is complete when the code runs, the reference checks make sense, you can explain every block without reading a solution, and you can modify at least one input while predicting the expected effect.